

CivicFlow AI Journey

Contents

01 CivicFlow-AI — 7:00–9:00 AM Interview Notes Today you c... RESPONSE

CivicFlow-AI — 7:00–9:00 AM Interview Notes

Today you covered the **Python + FastAPI + SQLAlchemy + PostgreSQL backend foundation** of CivicFlow-AI, including a complete CRUD API.

These notes are written for **placement/interview revision**, so I've included definitions, how things work, your project-specific explanation, and common interview questions.

1. Python Fundamentals

What is Python?

Python is a **high-level, interpreted, general-purpose programming language** known for its simple syntax and large ecosystem.

In CivicFlow-AI, Python is being used for:

- FastAPI backend
- AI/ML processing later
- Database interaction
- API development
- Business logic

Python vs Java — Important Interview Comparison

Java	Python
Statically typed	Dynamically typed
More verbose syntax	Concise syntax
Uses `{}` for blocks	Uses indentation
Semicolons commonly used	Semicolons generally unnecessary
JVM-based	Interpreted/bytecode execution via Python runtime
Spring Boot commonly used for backend	FastAPI commonly used in our project

Example:

Java

```
java
String name = "Aniket";
```

Python

```
python
name = "Aniket"
```

Python determines the type dynamically.

2. Python Dictionary

A dictionary stores data as **key-value pairs**.

Example from CivicFlow-AI:

```
python

report = {
    "title": "Broken Streetlight",
    "description": "Streetlight is not working",
    "location": "Ward 5",
    "severity": "High"
}
```

Accessing values:

```
python

report["title"]
report["location"]
```

Interview Question

Q: What is a dictionary in Python?

Answer:

"A dictionary is a mutable collection that stores data in key-value pairs. It is commonly used when data needs to be accessed using meaningful keys rather than numeric indexes."

3. Python List vs Dictionary

List

```
python

reports = ["Report 1", "Report 2", "Report 3"]
```

Access:

```
python

reports[0]
```

Dictionary

```
python

report = {
    "title": "Broken Streetlight",
    "location": "Ward 5"
}
```

Access:

```
python

report["title"]
```

Interview Question

Q: What is the difference between a list and dictionary?

"A list stores ordered elements accessed using indexes, while a dictionary stores key-value pairs accessed using keys."

4. Functions in Python

A function is defined using `def`.

```
python
def calculate_priority():
    if report["severity"] == "High":
        return "URGENT"
```

Calling:

```
python
priority = calculate_priority()
```

Parameter vs Argument

Parameter: variable defined in the function declaration.

```
python
def greet(name):
```

`name` is a parameter.

Argument: actual value passed during function call.

```
python
greet("Aniket")
```

`"Aniket"` is an argument.

Interview Question

Q: What is the purpose of a function?

"A function encapsulates reusable logic into a named block of code. It improves code reuse, readability, maintainability, and testing."

5. Python Indentation

Python uses **indentation to define code blocks**.

```
python
if severity == "High":
    print("Urgent")
```

The indented statement belongs to the `if` block.

Unlike Java, Python does not normally use `{ }` to define blocks.

6. FastAPI

What is FastAPI?

FastAPI is a **modern Python web framework used to build APIs and backend applications.**

CivicFlow-AI uses FastAPI as its backend framework.

Example:

```
python

from fastapi import FastAPI

app = FastAPI()
```

7. Uvicorn

What is Uvicorn?

Uvicorn is an **ASGI web server** used to run FastAPI applications.

We run CivicFlow-AI using:

```
powershell

python -m uvicorn app.main:app --reload
```

Breakdown:

```
text

python -m
↓
```

```
Run Python module

uvicorn
↓
ASGI server

app.main
↓
app/main.py

:app
↓
FastAPI application object

--reload
↓
Automatically reload during development
```

Interview Question

Q: Is Uvicorn a framework?

"No. FastAPI is the web framework, while Uvicorn is an ASGI server used to run the FastAPI application."

8. FastAPI Application

Our application:

```
python

from fastapi import FastAPI

app = FastAPI()
```

Here:

```
python

app = FastAPI()
```

creates the FastAPI application instance.

9. API Endpoint

An endpoint is a specific URL through which a client communicates with the backend.

Example:

```
python

@app.get("/api/health")
def health_check():
    return {
        "status": "CivicFlow-AI is Running..."
    }
```

This creates:

```
text

GET /api/health
```

10. GET vs POST

GET

Used primarily to **retrieve/read data**.

Example:

```
text

GET /api/reports
```

Means:

"Give me the reports."

POST

Used primarily to **send data/create a new resource**.

Example:

```
text

POST /api/reports
```

Means:

"Create a new report using this data."

11. PUT

PUT is commonly used to **update/replace an existing resource**.

Our endpoint:

```
text
PUT /api/reports/{report_id}
```

Example:

```
text
PUT /api/reports/1
```

Updates report ID `1`.

12. DELETE

DELETE is used to delete an existing resource.

Our endpoint:

```
text
DELETE /api/reports/{report_id}
```

Example:

```
text
DELETE /api/reports/3
```

Deletes report ID `3`.

13. HTTP CRUD Mapping

Remember this for interviews:

```
text
C → Create → POST
R → Read   → GET
U → Update → PUT
D → Delete → DELETE
```

Our CivicFlow API:

```
text
POST /api/reports
GET  /api/reports
GET  /api/reports/{id}
PUT  /api/reports/{id}
DELETE /api/reports/{id}
```

14. Request Body

A request body is the data sent by the client to the server as part of an HTTP request.

For example:

```
json

{
  "title": "Broken Streetlight",
  "description": "Streetlight is not working near school",
  "location": "Ward 5"
}
```

For our POST endpoint, this data is represented by:

```
python

report: ReportCreate
```

15. Pydantic

What is Pydantic?

Pydantic is used for **data validation, parsing, and defining data schemas** in our FastAPI application.

Example:

```
python

class ReportCreate(BaseModel):
    title: str
```

```
description: str
location: str
```

FastAPI uses this schema to validate incoming request data.

16. Pydantic `BaseModel`

We create schemas by inheriting from ``BaseModel``.

```
python

from pydantic import BaseModel

class ReportCreate(BaseModel):
    title: str
    description: str
    location: str
```

This tells FastAPI what structure the request should have.

17. Field Validation

We added constraints:

```
python

title: str = Field(
    min_length=3,
    max_length=50
)
```

Therefore:

```
text

Title < 3 characters
↓
Validation fails
```

and:

```
text

Title > 50 characters
↓
Validation fails
```

Similar validation exists for:

```
text

description → 10-500
location    → 2-100
```

18. Validation Flow

This is very important.

When a request arrives:

```
text

Client
↓
FastAPI Router
↓
Pydantic Validation
↓
Service / Business Logic
↓
Database
```

If the request is invalid:

```
text

Client
↓
FastAPI
↓
Pydantic
↓
Validation Error
↓
HTTP 422
```

The service does **not** need to process invalid request data.

Interview Question

Q: Where does request validation happen?

"In our FastAPI application, request data is validated by Pydantic schemas before the request reaches the business logic."

19. `ReportCreate`

Used for creating reports:

```
python

class ReportCreate(BaseModel):
    title: str
    description: str
    location: str
```

Flow:

```
text

POST
↓
ReportCreate
↓
Validation
↓
Router
↓
Service
↓
Database
```

20. `ReportUpdate`

Used for PUT requests:

```
python

class ReportUpdate(BaseModel):
    title: str
    description: str
    location: str
```

Flow:

```
text

PUT /reports/1
↓
ReportUpdate
↓
Validation
↓
Router
↓
Service
↓
Database
```

We keep ``ReportCreate`` and ``ReportUpdate`` separate even though their fields currently match because their purposes are different and their validation requirements can evolve independently.

21. Response Schema

We created:

```
python
```

```
class ReportResponse(BaseModel):
    id: int
    title: str
    description: str
    location: str

    model_config = ConfigDict(from_attributes=True)
```

This defines what the API returns.

22. `response\_model`

Example:

```
python

@router.get(
    "/reports/{report_id}",
    response_model=ReportResponse
)
```

`response\_model` tells FastAPI:

"The response should follow this Pydantic schema."

Benefits:

- Response validation
- Serialization
- API documentation
- Consistent response structure

Interview Answer

""`response\_model` defines and validates the structure of data returned by a FastAPI endpoint. It also helps FastAPI generate accurate OpenAPI documentation.""

23. `from\_attributes=True`

We return SQLAlchemy objects from our service.

Pydantic needs to know that it can read values from object attributes.

Therefore:

```
python

model_config = ConfigDict(from_attributes=True)
```

allows Pydantic to convert the SQLAlchemy ORM object into the response schema.

Interview Answer

""`from\_attributes=True` allows Pydantic to construct a model from object attributes, which is useful when working with SQLAlchemy ORM objects.""

24. SQLAlchemy

What is SQLAlchemy?

SQLAlchemy is a Python SQL toolkit and ORM.

In CivicFlow-AI, we use it to:

- Connect Python to PostgreSQL
- Define database models
- Query records
- Insert records
- Update records
- Delete records
- Manage database sessions

25. ORM

ORM means:

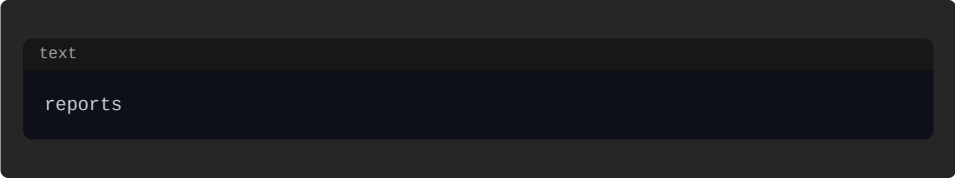
“Object-Relational Mapping”

It maps programming language objects/classes to relational database tables.

Our:

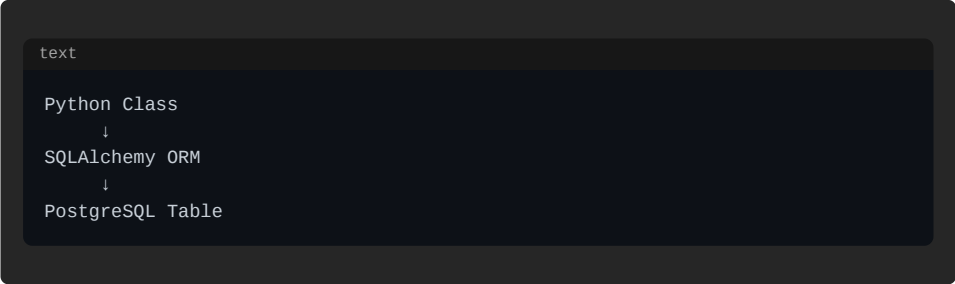
```
python
class Report(Base):
```

represents the:



database table.

Conceptually:



26. SQLAlchemy Model

Our model:

```
python
class Report(Base):
    __tablename__ = "reports"

    id: Mapped[int] = mapped_column(
        Integer,
        primary_key=True,
        index=True
    )

    title: Mapped[str] = mapped_column(
        String(100),
        nullable=False
    )

    description: Mapped[str] = mapped_column(
        String(500),
        nullable=False
    )

    location: Mapped[str] = mapped_column(
        String(100),
```

```
        nullable=False
    )
```

27. `Base`

We created:

```
python

class Base(DeclarativeBase):
    pass
```

`Base` is the base class used by SQLAlchemy declarative models.

Our `Report` inherits from it:

```
python

class Report(Base):
```

This allows SQLAlchemy to recognize `Report` as a mapped database model.

28. `\_\_tablename\_\_`

```
python
```

```
__tablename__ = "reports"
```

Specifies the corresponding database table name.

Therefore:

```
text

Report class
  ↓
reports table
```

29. Primary Key

```
python

id: Mapped[int] = mapped_column(
    Integer,
    primary_key=True
)
```

The `id` uniquely identifies each report.

Example:

```
text

1 → Report A
2 → Report B
3 → Report C
```

When a new POST creates a report, PostgreSQL generates a new ID.

That's why submitting the same report multiple times resulted in:

```
text
1
2
3
```

They were three separate database rows.

30. `nullable=False`

Example:

```
python

title = mapped_column(
    String(100),
    nullable=False
)
```

This means the database column cannot contain `NULL`.

31. Engine

We created:

```
python

engine = create_engine(DATABASE_URL)
```

The SQLAlchemy **Engine** manages the infrastructure for communicating with the database.

Think:

```
text

Engine
↓
Database connectivity
```

Interview Answer

“The SQLAlchemy Engine is the central object responsible for managing database connectivity and connection pooling.”

32. Session

We created:

```
python

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

A Session provides a working context for performing database operations and transactions.

Conceptually:

```
text

Engine
↓
Session
↓
Query / Insert / Update / Delete
```

33. `sessionmaker`

`sessionmaker` is a **factory for creating SQLAlchemy Session objects**.

```
python

SessionLocal = sessionmaker(bind=engine)
```

Then:

```
python

db = SessionLocal()
```

creates a session.

34. `get\_db()`

Our dependency:

```
python

def get_db():
    db = SessionLocal()

    try:
        yield db
    finally:
        db.close()
```

Purpose:

1. Create database session
2. Give it to the endpoint
3. Perform database operations
4. Close session afterward

35. `Depends(get\_db)`

In our router:

```
python

db: Session = Depends(get_db)
```

This is FastAPI's **dependency injection mechanism**.

FastAPI calls `get\_db()` and injects the resulting database session into the endpoint.

Interview Answer

```
"""`depends()` is FastAPI's dependency injection mechanism. In our application,
`depends(get_db)` provides a database session to the endpoint and ensures the session is closed
after the request."""
```

36. Database Session Lifecycle

Remember:

```
text
Request
↓
get_db()
↓
Session created
↓
Endpoint
↓
Service
↓
Database operation
↓
finally
↓
Session closed
```

37. `db.add()`

When creating a report:

```
python
db.add(report)
```

This adds the SQLAlchemy object to the current session.

It stages the object for persistence.

It does **not by itself mean the transaction has been committed**.

38. `db.commit()`

```
python
db.commit()
```

Commits the current transaction.

For creation/update/deletion, this makes the database change persistent.

Interview Answer

```
"""`commit()` commits the current transaction and persists the pending database changes."""
```

39. `db.refresh()`

After creating:

```
python
db.refresh(report)
```

refreshes the SQLAlchemy object with the current database state.

This is useful for database-generated values such as:

```
text
id
```

40. `db.delete()`

For deletion:

```
python
db.delete(report)
db.commit()
```

`db.delete()` marks the object for deletion in the current session.

`commit()` persists the deletion.

41. Database Table

Our PostgreSQL database:

```
text
civicflow
```

contains:

```
text
reports
```

with:

```
text
id
title
description
location
```

42. PostgreSQL

PostgreSQL is the relational database used by CivicFlow-AI.

Our architecture:

```
text
FastAPI
↓
```

```
SQLAlchemy
↓
psycopg2
↓
PostgreSQL
```

``psycopg2-binary`` provides the PostgreSQL driver used by SQLAlchemy in our setup.

43. CRUD Service Logic

Create

```
python

report = Report(...)
db.add(report)
db.commit()
db.refresh(report)
```

Read

```
python

db.query(Report).all()
```

Read One

```
python
```

```
db.query(Report).filter(
    Report.id == report_id
).first()
```

Update

```
python

report.title = title
report.description = description
report.location = location

db.commit()
db.refresh(report)
```

Delete

```
python

db.delete(report)
db.commit()
```

44. ``.first()``

Example:

```
python
```

```
report = db.query(Report).filter(
    Report.id == report_id
).first()
```

This returns the first matching result or `None` if nothing matches.

That's why we use:

```
python

if report is None:
```

45. HTTP 404

If the requested report doesn't exist:

```
python

raise HTTPException(
    status_code=404,
    detail="Report not found"
)
```

Example:

```
text

GET /api/reports/999
```

Response:

```
json

{
  "detail": "Report not found"
}
```

46. HTTP 422

When Pydantic validation fails, FastAPI returns a validation error, commonly HTTP:

```
text

422 Unprocessable Entity
```

Example:

```
json

{
  "title": "A",
  "description": "Short",
  "location": "Ward 5"
}
```

The schema rejects it because the fields don't satisfy the configured constraints.

47. Router Layer

Our router contains endpoints such as:

```
python

@router.post("/reports")
```

Responsibilities:

- Define API routes
- Receive HTTP requests
- Receive validated schema objects
- Obtain dependencies
- Call service functions
- Return responses

The router should **not contain large amounts of business logic**.

48. Service Layer

Example:

```
python

def create_report(...):
```

The service contains application/business logic.

For example:

```
text

Router
↓
create_report()
↓
Create Report object
↓
db.add()
↓
db.commit()
```

This keeps the router clean.

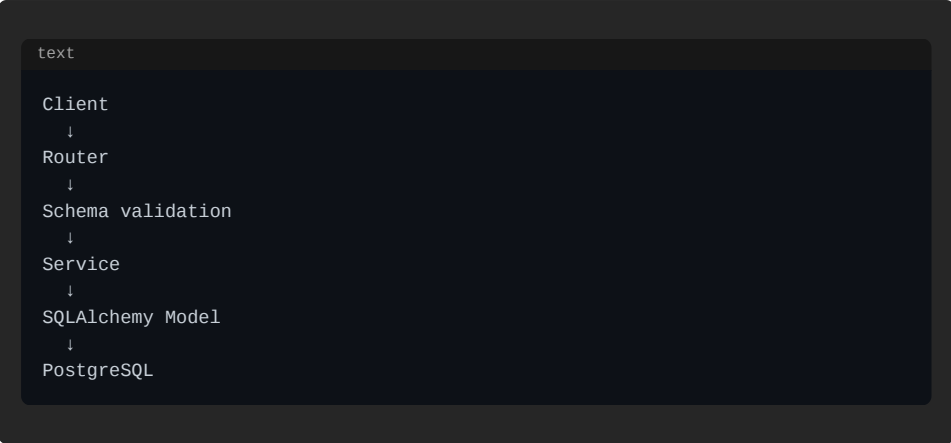
49. Layered Architecture

Our CivicFlow-AI backend:

```
text

backend/
├── app/
│   ├── database/
│   │   └── database.py
│   ├── models/
│   │   └── report_model.py
│   ├── routers/
│   │   └── report_router.py
│   ├── schemas/
│   │   └── report_schema.py
│   ├── services/
│   │   └── report_service.py
│   └── main.py
```

Flow:



50. Why Layered Architecture?

The main reason is **separation of concerns**.
Instead of putting everything inside `main.py`, responsibilities are separated.

- Benefits:
- Maintainability
 - Readability
 - Reusability
 - Testing
 - Easier debugging
 - Easier team development
 - Easier future expansion

51. `main.py`

Our `main.py` registers the router:

```
python
app.include_router(
    report_router.router,
    prefix="/api"
)
```

This means routes defined as:

```
python
@router.get("/reports")
```

become:

because of the `/api` prefix.

52. FastAPI vs Spring Boot

Since you already know Spring Boot, this comparison is extremely useful in interviews.

Spring Boot	CivicFlow-AI / FastAPI
Controller	Router

Spring Boot	CivicFlow-AI / FastAPI
DTO	Pydantic Schema
Service	Service
Entity	SQLAlchemy Model
JPA/Hibernate	SQLAlchemy ORM
Repository	SQLAlchemy Session/query
<code>@GetMapping`</code>	<code>@router.get()`</code>
<code>@PostMapping`</code>	<code>@router.post()`</code>
<code>@PutMapping`</code>	<code>@router.put()`</code>
<code>@DeleteMapping`</code>	<code>@router.delete()`</code>
Dependency Injection	<code>@depends()`</code>
MySQL in your previous project	PostgreSQL here

Important: These are conceptual equivalents, not exact implementations.

53. Complete CivicFlow-AI Request Flow

This is one of the most important things to remember for your interview.

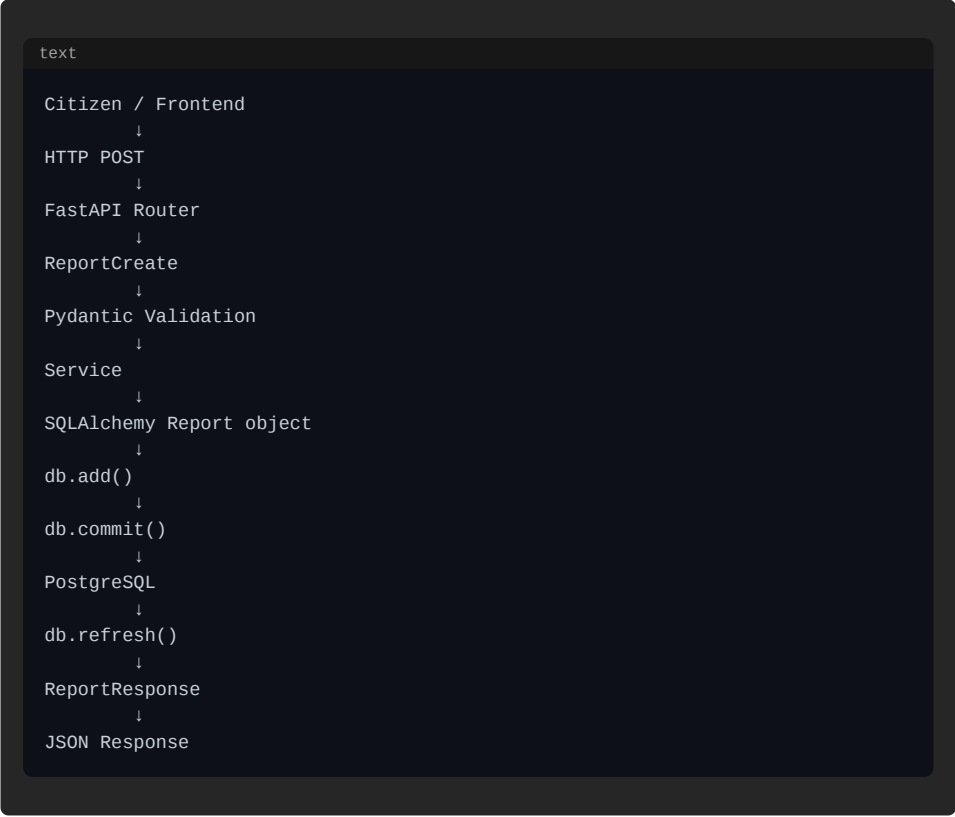
Suppose a citizen submits:

```
http
POST /api/reports
```

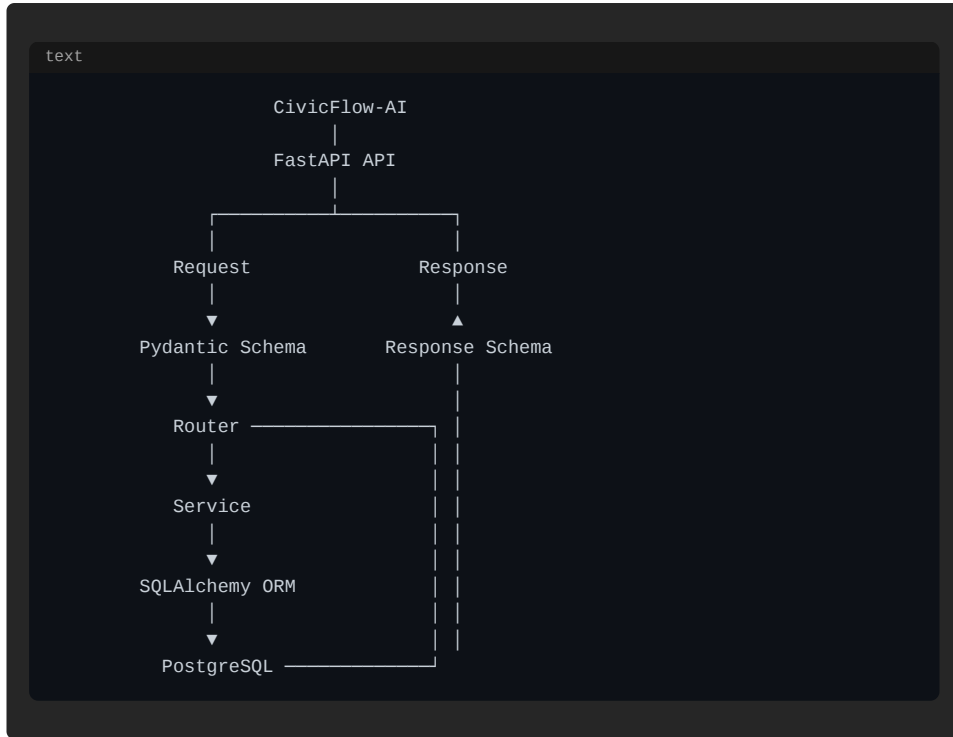
with:

```
json
{
  "title": "Broken Streetlight",
  "description": "Streetlight is not working near school",
  "location": "Ward 5"
}
```

The flow is:



54. Complete CRUD Architecture



Top Interview Questions From Today's Session

1. What is FastAPI?

"FastAPI is a modern Python web framework for building APIs and backend applications."

2. What is Uvicorn?

"Uvicorn is an ASGI server used to run FastAPI applications."

3. What is Pydantic?

"Pydantic is used for data validation, parsing, and defining schemas in FastAPI applications."

4. What is `BaseModel``?

"`BaseModel`` is the base class used by Pydantic to define structured and validated data models."

5. What is SQLAlchemy?

"SQLAlchemy is a Python SQL toolkit and ORM that allows applications to interact with relational databases using Python objects and queries."

6. What is ORM?

"ORM stands for Object-Relational Mapping. It maps programming-language objects/classes to relational database tables."

7. What is an Engine?

"The SQLAlchemy Engine manages database connectivity and connection pooling."

8. What is a Session?

"A Session provides a working context for database operations and transaction management."

9. What does ``Depends()``` do?

"``Depends()``` provides FastAPI's dependency injection mechanism. We use it to inject a database session into API endpoints."

10. What does ``db.commit()``` do?

"It commits the current transaction and makes pending database changes persistent."

11. Why use ``db.refresh()```?

"It refreshes the SQLAlchemy object with the current database state, including database-generated values such as IDs."

12. Difference between POST and PUT?

"POST is commonly used to create a new resource, while PUT is commonly used to update or replace an existing resource."

13. What is `response\_model`?

"It defines and validates the structure of the response returned by a FastAPI endpoint and contributes to automatic API documentation."

14. Why use separate schemas?

"Separate request and response schemas provide clear API contracts and allow their validation and fields to evolve independently."

15. Why use layered architecture?

"It separates responsibilities between routing, validation, business logic, database mapping, and persistence, improving maintainability and testability."

16. What happens when a report ID doesn't exist?

"The service checks whether the query returned `None` and raises an HTTP 404 exception with `\"Report not found\"`."

17. What happens when invalid request data is sent?

"Pydantic validation fails before the business logic executes, and FastAPI returns a validation error, commonly HTTP 422."

18. Explain your CivicFlow-AI backend.

Strong interview answer:

"CivicFlow-AI uses FastAPI as the backend framework and PostgreSQL as the database. I use Pydantic schemas for request validation and response models, SQLAlchemy as the ORM, and a layered architecture consisting of routers, schemas, services, models, and database configuration."

I implemented complete CRUD operations for civic reports. FastAPI's dependency injection provides a database session per request, while the service layer handles database operations such as create, read, update, and delete."



30-Second Revision

Before an interview, remember this:

```
text

FastAPI
→ Backend/API framework

Uvicorn
→ Runs FastAPI

Pydantic
→ Validation + schemas

Router
→ API endpoints

Service
→ Business logic

SQLAlchemy
→ ORM/database interaction

Engine
→ Database connectivity

Session
→ Database operation/transaction context

PostgreSQL
→ Relational database

Depends()
→ Dependency Injection

commit()
→ Persist transaction
```

```
refresh()
→ Refresh ORM object

CRUD
→ POST / GET / PUT / DELETE

response_model
→ Response contract

404
→ Resource not found

422
→ Validation error
```

Today's achievement

You didn't just learn individual Python/FastAPI concepts. You built the complete backend foundation:

```
text

Python
↓
FastAPI
↓
Pydantic
↓
SQLAlchemy
↓
PostgreSQL
↓
Layered Architecture
↓
CRUD API
↓
Response Validation
```

That is the **7-9 AM CivicFlow-AI session's complete interview revision set.**